

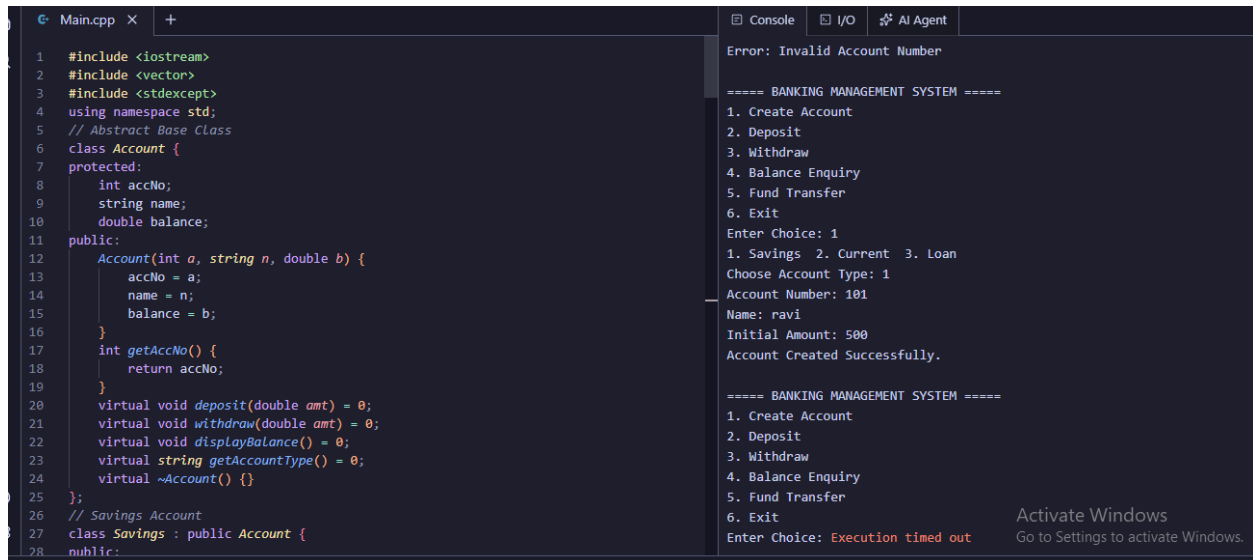
CO5-AT2-ASSESSMENT 1

V. Sree Pavithra Reddy
192465020

1. Develop a Banking Management System that supports Savings, Current, and Loan Accounts. The system must allow users to perform operations such as account creation, deposit, withdrawal, balance enquiry, and loan processing.(CO5)

Requirements:

1. Define an abstract base class Account with virtual and pure virtual functions for common operations (deposit, withdraw, displayBalance, getAccountType).
2. Use runtime polymorphism (base class pointers/references) so the correct account-specific behavior executes based on the actual object type.
3. Handle errors using exception handling — insufficient balance, invalid account number, negative deposit, and invalid withdrawal.
4. Demonstrate throwing, catching, and rethrowing exceptions during a transaction (e.g., a failed fund transfer).
5. Use exception specifications (throw(...)) on functions to declare the exceptions they may raise.



```
1 #include <iostream>
2 #include <vector>
3 #include <stdexcept>
4 using namespace std;
5 // Abstract Base Class
6 class Account {
7 protected:
8     int accNo;
9     string name;
10    double balance;
11 public:
12    Account(int a, string n, double b) {
13        accNo = a;
14        name = n;
15        balance = b;
16    }
17    int getAccNo() {
18        return accNo;
19    }
20    virtual void deposit(double amt) = 0;
21    virtual void withdraw(double amt) = 0;
22    virtual void displayBalance() = 0;
23    virtual string getAccountType() = 0;
24    virtual ~Account() {}
25 };
26 // Savings Account
27 class Savings : public Account {
28 public:
```

Console

Error: Invalid Account Number

===== BANKING MANAGEMENT SYSTEM =====

1. Create Account
2. Deposit
3. Withdraw
4. Balance Enquiry
5. Fund Transfer
6. Exit

Enter Choice: 1
1. Savings 2. Current 3. Loan
Choose Account Type: 1
Account Number: 101
Name: ravi
Initial Amount: 500
Account Created Successfully.

===== BANKING MANAGEMENT SYSTEM =====

1. Create Account
2. Deposit
3. Withdraw
4. Balance Enquiry
5. Fund Transfer
6. Exit

Enter Choice: Execution timed out

Activate Windows
Go to Settings to activate Windows.

```
11 void withdraw(double amt) override {
12     balance -= amt;
13     cout << "Withdrawal Successful.\n";
14 }
15 void displayBalance() override {
16     cout << "Current Balance: " << balance << endl;
17 }
18 string getAccountType() override {
19     return "Current";
20 }
21 };
22 // Loan Account
23 class Loan : public Account {
24 public:
25     Loan(int a, string n, double b) : Account(a, n, b) {}
26     void deposit(double amt) override {
27         if (amt <= 0)
28             throw runtime_error("Negative Deposit");
29         balance += amt;
30         cout << "Loan Repaid Successfully.\n";
31     }
32     void withdraw(double amt) override {
33         if (amt <= 0)
34             throw runtime_error("Invalid Loan Amount");
35         balance -= amt;
36         cout << "Loan Processed Successfully.\n";
37     }
38 };
39
40 Error: Invalid Account Number
41
42 ===== BANKING MANAGEMENT SYSTEM =====
43 1. Create Account
44 2. Deposit
45 3. Withdraw
46 4. Balance Enquiry
47 5. Fund Transfer
48 6. Exit
49 Enter Choice: 1
50 1. Savings 2. Current 3. Loan
51 Choose Account Type: 1
52 Account Number: 101
53 Name: ravi
54 Initial Amount: 500
55 Account Created Successfully.
56
57 ===== BANKING MANAGEMENT SYSTEM =====
58 1. Create Account
59 2. Deposit
60 3. Withdraw
61 4. Balance Enquiry
62 5. Fund Transfer
63 6. Exit
64 Enter Choice: Execution timed out
65
66 Activate Windows
67 Go to Settings to activate Windows.
```

```
1 int main() {
2     vector<Account*> accounts;
3     int choice;
4     do {
5         cout << "\n===== BANKING MANAGEMENT SYSTEM =====\n";
6         cout << "1. Create Account\n";
7         cout << "2. Deposit\n";
8         cout << "3. Withdraw\n";
9         cout << "4. Balance Enquiry\n";
10        cout << "5. Fund Transfer\n";
11        cout << "6. Exit\n";
12        cout << "Enter Choice: ";
13        cin >> choice;
14
15        try {
16            if (choice == 1) {
17                int type, no;
18                string name;
19                double bal;
20
21                cout << "1. Savings 2. Current 3. Loan\n";
22                cout << "Choose Account Type: ";
23                cin >> type;
24
25                cout << "Account Number: ";
26                cin >> no;
27
28                cout << "Name: ";
29
30            }
31
32            Error: Invalid Account Number
33
34            ===== BANKING MANAGEMENT SYSTEM =====
35            1. Create Account
36            2. Deposit
37            3. Withdraw
38            4. Balance Enquiry
39            5. Fund Transfer
40            6. Exit
41            Enter Choice: 1
42            1. Savings 2. Current 3. Loan
43            Choose Account Type: 1
44            Account Number: 101
45            Name: ravi
46            Initial Amount: 500
47            Account Created Successfully.
48
49            ===== BANKING MANAGEMENT SYSTEM =====
50            1. Create Account
51            2. Deposit
52            3. Withdraw
53            4. Balance Enquiry
54            5. Fund Transfer
55            6. Exit
56            Enter Choice: Execution timed out
57
58            Activate Windows
59            Go to Settings to activate Windows.
```

```
119 int main() {
120     do {
121         try {
122             else if (choice == 5) {
123                 transfer(findAccount(accounts, from),
124
125             }
126
127             else if (choice == 6) {
128                 cout << "Thank You!\n";
129             }
130
131             else {
132                 cout << "Invalid Choice.\n";
133             }
134
135         } catch (runtime_error& e) {
136             cout << "Error: " << e.what() << endl;
137         }
138     } while (choice != 6);
139
140     // Free Memory
141     for (auto acc : accounts)
142         delete acc;
143
144     return 0;
145 }
146
147 Error: Invalid Account Number
148
149 ===== BANKING MANAGEMENT SYSTEM =====
150 1. Create Account
151 2. Deposit
152 3. Withdraw
153 4. Balance Enquiry
154 5. Fund Transfer
155 6. Exit
156 Enter Choice: 1
157 1. Savings 2. Current 3. Loan
158 Choose Account Type: 1
159 Account Number: 101
160 Name: ravi
161 Initial Amount: 500
162 Account Created Successfully.
163
164 ===== BANKING MANAGEMENT SYSTEM =====
165 1. Create Account
166 2. Deposit
167 3. Withdraw
168 4. Balance Enquiry
169 5. Fund Transfer
170 6. Exit
171 Enter Choice: Execution timed out
172
173 Activate Windows
174 Go to Settings to activate Windows.
```

2. Develop an Online Payment System for an e-commerce application that supports Credit Card, Debit Card, UPI, and Net Banking payments. The system should process different payment methods and handle transaction failures safely.(CO5)

Requirements:

1. Define an abstract base class `PaymentMethod` with virtual and pure virtual functions forming a common payment interface (`validateDetails`, `pay`, `getMethodNames`).
2. Use runtime polymorphism to dynamically select and execute the correct payment method's logic at runtime.
3. Handle errors using exception handling — invalid card details, insufficient funds, incorrect UPI PIN, and failed transactions.
4. Demonstrate throwing, catching, and rethrowing exceptions during payment processing (e.g., in a payment gateway's checkout flow).
5. Use exception specifications (`throw(...)`) to declare possible payment-related exceptions on each function.
6. Display appropriate success or error messages based on the transaction outcome.

```
1  #include <iostream>
2  #include <stdexcept>
3  using namespace std;
4  // Abstract Base Class
5  class PaymentMethod {
6  public:
7      virtual void validateDetails() = 0;
8      virtual void pay(double amount) = 0;
9      virtual string getMethodName() = 0;
10     virtual ~PaymentMethod() {}
11 };
12 // Credit Card
13 class CreditCard : public PaymentMethod {
14     string card;
15 public:
16     CreditCard(string c) {
17         card = c;
18     }
19     void validateDetails() override {
20         if (card.Length() != 16)
21             throw runtime_error("Invalid Credit Card Number");
22     }
23     void pay(double amount) override {
24         validateDetails();
25         if (amount > 50000)
26             throw runtime_error("Insufficient Credit Limit");
27         cout << "Credit Card Payment Successful." << endl;
28     }

```

```
===== ONLINE PAYMENT SYSTEM =====
1. Credit Card
2. Debit Card
3. UPI
4. Net Banking
Enter Choice: 3
Enter Amount: 500
Enter UPI PIN: 1234
UPI Payment Successful.
Checkout Completed Successfully.
```

Activate Windows
Go to Settings to activate Win

```

36 public:
37     {
43     void pay(double amount) override {
44         validateDetails();
45         if (balance < amount)
46             throw runtime_error("Insufficient Funds");
47         balance -= amount;
48         cout << "Debit Card Payment Successful." << endl;
49     }
50     string getMethodName() override {
51         return "Debit Card";
52     }
53 };
54 // UPI
55 class UPI : public PaymentMethod {
56     int pin;
57 public:
58     UPI(int p) {
59         pin = p;
60     }
61     void validateDetails() override {
62         if (pin != 1234)
63             throw runtime_error("Incorrect UPI PIN");
64     }
65     void pay(double amount) override {
66         validateDetails();
67         cout << "UPI Payment Successful." << endl;
68     }
69     string getMethodName() override {

```

```

===== ONLINE PAYMENT SYSTEM =====
1. Credit Card
2. Debit Card
3. UPI
4. Net Banking
Enter Choice: 3
Enter Amount: 500
Enter UPI PIN: 1234
UPI Payment Successful.
Checkout Completed Successfully.

```

Activate Windows
Go to Settings to activate Windows.

```

cout << "3. UPI" << endl;
cout << "4. Net Banking" << endl;
cout << "Enter Choice: ";
cin >> choice;
cout << "Enter Amount: ";
cin >> amount;
try {
    if (choice == 1) {
        string card;
        cout << "Enter 16-digit Card Number: ";
        cin >> card;
        payment = new CreditCard(card);
    }
    else if (choice == 2) {
        double bal;
        cout << "Enter Account Balance: ";
        cin >> bal;
        payment = new DebitCard(bal);
    }
    else if (choice == 3) {
        int pin;
        cout << "Enter UPI PIN: ";
        cin >> pin;
        payment = new UPI(pin);
    }
    else if (choice == 4) {
        bool active;

```

```

===== ONLINE PAYMENT SYSTEM =====
1. Credit Card
2. Debit Card
3. UPI
4. Net Banking
Enter Choice: 3
Enter Amount: 500
Enter UPI PIN: 1234
UPI Payment Successful.
Checkout Completed Successfully.

```

Activate Windows
Go to Settings to activate Windows.